Studio Nx

JS 02 — apiFetch()

2026-10-07

JS 02 — `apiFetch(path, options)`

항목	내용
파일	`studionx/frontend/static/js/core/api.js` (도우미 `toApiError` 포함)
작업	P1-02 프론트 공통 JS · PRD 01 FR-105
작성	Claude (AI) · 2026-10-06 · 사용자 검토
이전 · 다음	01 `refreshAccessToken()` · 03 `safeNextPath()`

1. 하는 일

서버 API 를 부르는 **하나뿐인** 함수. 토큰을 붙이고, 만료되면 재발급 후 한 번 다시 보내고, 실패는 모두 `ApiError` 로 바꿔 던진다.

2. 언제 쓰나

모든 페이지 · 모듈은 `fetch` 를 직접 쓰지 않고 이 함수(또는 짧은 이름 `api.get` · `api.post` …)를 쓴다.

예	부르는 곳
`api.post('/auth/login', { email, password, remember }, { auth: false })`	로그인 모달 (P1-07)
`api.get('/users/me')`	회원 정보 · 헤더 (P1-10)
`api.post('/users/me/image', formData)`	프로필 이미지 업로드 (P1-10)
`api.get('/projects?page=1')` · `api.post('/projects', { name })`	내 프로젝트 (P1-14)
`api.get('/me/dashboard')`	마이페이지 (P1-17)

옵션	기본	뜻
`method`	`'GET'`	HTTP 방식
`body`	없음	객체 → JSON, `FormData` → 그대로 (파일 업로드)
`auth`	`true`	`false` 면 토큰을 붙이지 않음 — 회원가입 · 로그인 · 문의처럼 로그인 전에 쓰는 API

기대는 원리

원리	내용
`Authorization: Bearer <Access Token>`	서버가 "누가 보냈는지" 아는 방법. 로그인이 필요한 API 는 이 헤더가 없거나 토큰이 만료되면 **401** `UNAUTHORIZED`
서버 오류 형식	모든 실패가 `{ "error": { "code", "message", "details" } }` (P1-03 `core/errors.py`) → 화면은 `err.code` 로 분기, `err.message` 를 보여 줌
`credentials: 'same-origin'`	같은 사이트 요청에 쿠키를 싣는다 — 리프레시 쿠키는 `/api/v1/auth` 아래에만 실림 (01)

3. 코드

import (파일 맨 위) `api.js` 는 다른 파일을 import 하지 않는다 (공통 모듈의 맨 아래 단계). 이 함수가 쓰는 `accessToken · refreshAccessToken · ApiError · toApiError · API_BASE` 는 같은 파일 안에 있고, `fetch` 는 브라우저 기본 기능이다.

함수

```
export async function apiFetch(path, options = {}) {
  const { method = 'GET', body, auth = true } = options;

  const headers = { Accept: 'application/json' };
  let payload;
  if (body instanceof FormData) {
    payload = body;
  } else if (body !== undefined && body !== null) {
    headers['Content-Type'] = 'application/json';
    payload = JSON.stringify(body);
  }

  if (auth && !accessToken) await refreshAccessToken();

  const send = async () => {
    const h = auth && accessToken ? { ...headers, Authorization: `Bearer ${accessToken}` } : headers;
    try {
      return await fetch(API_BASE + path, { method, headers: h, body: payload, credentials: 'same-origi
n' });
    } catch {
      throw new ApiError(0, 'NETWORK_ERROR', 'Network error. Please try again.');
    }
  };

  let res = await send();

  if (res.status === 401 && auth) {
    const err = await toApiError(res);
    if (err.code !== 'UNAUTHORIZED' || !(await refreshAccessToken())) throw err;
    res = await send();
  }

  if (!res.ok) throw await toApiError(res);

  if (res.status === 204) return null;
  const text = await res.text();
  return text ? JSON.parse(text) : null;
}

async function toApiError(res) {
  const data = await res.json().catch(() => null);
  const e = data?.error;
  if (e?.code) return new ApiError(res.status, e.code, e.message ?? '', e.details ?? null);
  return new ApiError(res.status, 'INTERNAL_ERROR', 'Something went wrong. Please try again.');
}
```

문법 풀이

import 문법	뜻
`export` (함수 앞)	다른 파일이 `import { 함수 }` 로 가져다 쓸 수 있게 내보냄. 없으면 이 파일 안에서만 쓸 수 있음

식	뜻
`const { method = 'GET', body, auth = true } = options`	**구조 분해 + 기본값**: `options.method` 가 없으면 `'GET'`, `options.auth` 가 없으면 `true`
`body instanceof FormData`	`body` 가 `FormData` 로 만든 값인지 — 파일 업로드인지 확인
`{ ...headers, Authorization: … }`	**펼침**: `headers` 를 복사하고 한 칸 더한 새 객체. 원래 `headers` 는 그대로라 재시도 때 새 토큰으로 다시 만들 수 있음
`auth && accessToken ? A : B`	둘 다 참이면 A(토큰 붙임), 아니면 B(그대로)
`if (err.code !== 'UNAUTHORIZED' || !(await refreshAccessToken())) throw err`	"만료 오류가 아니거나, 재발급이 실패하면 던진다". `||` 는 앞이 참이면 뒤를 실행하지 않음 → 다른 401(예: 로그인 실패)이면 **재발급을 시도조차 안 함**
`e.message ?? ''`	`??`: 왼쪽이 `null`·`undefined` 일 때만 오른쪽. (`||` 와 달리 `''` · `0` 은 그대로 둠)
`text ? JSON.parse(text) : null`	본문이 비었으면 `null` — `res.json()` 은 빈 본문에서 오류가 나서 글자로 먼저 읽음

4. 흐름

1. 옵션을 읽고 본문을 만든다 (객체 → JSON 문자열, `FormData` → 그대로).
2. 로그인이 필요한 요청인데 토큰이 없으면 먼저 재발급.
3. 보낸다 — 토큰이 있으면 `Authorization` 헤더. 네트워크가 끊기면 `NETWORK_ERROR`(status 0).
4. 401 `UNAUTHORIZED` 면 재발급 → 성공하면 **한 번만** 다시 보낸다. 재발급 실패 · 다른 401 이면 그 오류를 던진다.
5. 그 밖의 실패(4xx · 5xx)는 서버 형식대로 `ApiError`. JSON 이 아니면 `INTERNAL_ERROR`.
6. 성공: 204 · 빈 본문 → `null`, 아니면 JSON.

5. 왜 이렇게

선택	이유
fetch 를 한 곳에서만	토큰 · 재발급 · 오류 형식을 페이지마다 다시 쓰지 않음
`FormData` 면 `Content-Type` 을 안 붙임	브라우저가 `multipart/form-data; boundary=…` 를 직접 붙여야 서버가 파일을 읽음. 직접 쓰면 boundary 가 빠져 깨짐
재시도는 한 번만	재발급 뒤에도 401 이면 무한 반복 대신 바로 실패
401 이라도 `UNAUTHORIZED` 만 재발급	로그인 실패(`INVALID_CREDENTIALS`) 같은 401 은 토큰 문제가 아님
`send()` 안에서 헤더를 매번 만듦	재시도 때 **새** 토큰이 실림
실패를 모두 `ApiError` 로	화면은 `try { … } catch (err) { if (err.code === 'EMAIL_TAKEN') … }` 한 가지 방식으로 처리
실패 본문을 `toApiError` 로 분리	401 처리와 일반 실패가 같은 변환을 씀

6. 시험 (fetch 를 가짜로)

경우	결과
토큰 없음 → `/users/me`	refresh → 요청 순서, `Bearer T1` · `Accept` · `credentials`, 결과 JSON
객체 body (POST)	`Content-Type: application/json`, 본문 `{"name":"A"}`
`FormData` body	그대로, `Content-Type` 안 붙음
`auth: false`	재발급 안 함, `Authorization` 없음
401 `UNAUTHORIZED`	refresh → 같은 요청을 새 토큰으로, 성공 (요청 3번)
401 + 재발급 실패	`ApiError(401, UNAUTHORIZED)`, 토큰 지움
재시도도 401	한 번만 재시도 후 던짐 (요청 3번)
401 `INVALID_CREDENTIALS`	재발급 없이 그대로 던짐 (요청 1번)
422	`code` · `message` · `details.email` 그대로
500 HTML 응답	`INTERNAL_ERROR`
네트워크 오류	status 0 · `NETWORK_ERROR`
204 · 빈 본문 200	`null`